

AutoML Agent System — Project Report

Course: Software Engineering
Project: Multi-Agent AutoML Pipeline
Repository: Dhruba531/automl-agent-system
Branch: claude/swe-course-project-udo02u
Date: June 2026

1. Introduction

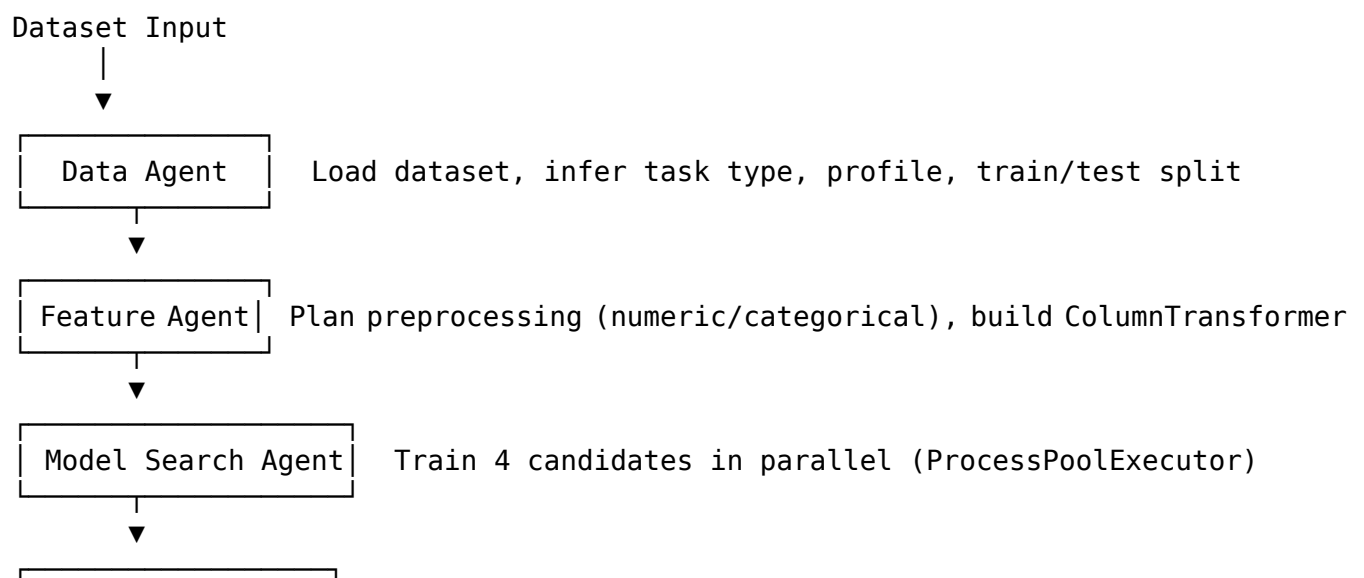
This project implements a fully automated machine learning (AutoML) pipeline as a multi-agent system. The design is inspired by the research paper AutoML-Agent: A Multi-Agent LLM Framework for Full-Pipeline AutoML (arXiv:2410.02958). The system takes a raw tabular dataset as input and produces a trained, evaluated, explained, monitored, and deployed model as output — with no manual intervention between steps.

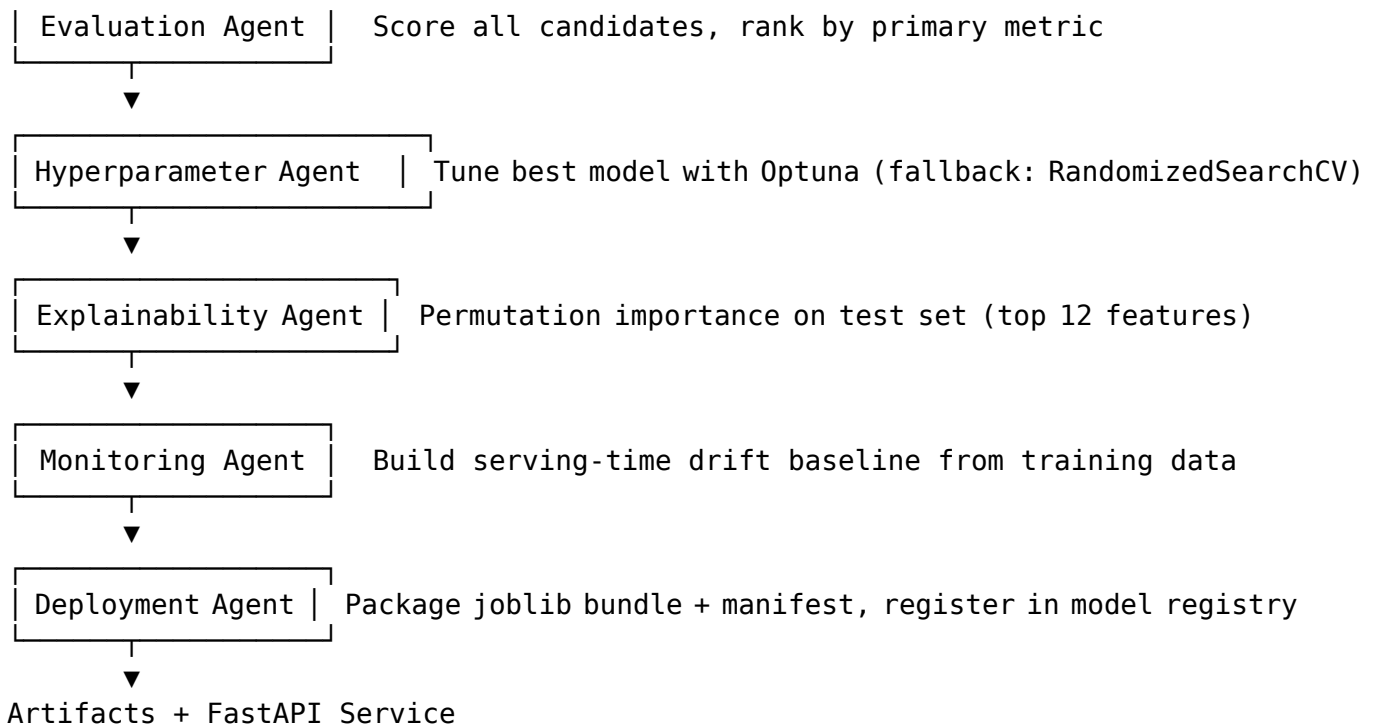
The primary goal was to demonstrate production-oriented software engineering principles (SOLID, DRY, KISS, YAGNI) within a realistic ML engineering context, and to deliver a working end-to-end system complete with a REST API and browser console UI.

2. System Architecture

2.1 Multi-Agent Pipeline

The system is divided into **8 specialised agents**, each with a single, well-defined responsibility. They are coordinated by a central **Orchestrator** that sequences their execution without coupling them to each other.





2.2 Module Structure

```

automl_agent/
├── agents/                8 pipeline agents + shared BaseAgent
│   ├── base.py
│   ├── data.py
│   ├── feature.py
│   ├── model_search.py
│   ├── evaluation.py
│   ├── hyperparameter.py
│   ├── explainability.py
│   ├── monitoring.py
│   └── deployment.py
├── serving/              FastAPI REST API + Google OAuth
│   ├── app.py
│   ├── auth.py
│   ├── config.py
│   ├── model_store.py
│   └── schemas.py
├── frontend/            Browser console UI (HTML/CSS/JS, no build step)
│   ├── index.html
│   ├── styles.css
│   └── app.js
├── orchestrator.py      Pipeline coordinator
├── harness.py           Repeatable experiment runner
├── registry.py          Append-only model version registry
└── types.py             Shared dataclasses and type aliases
  
```

└─ cli.py Command-line interface

Total source: ~1,634 lines Python · ~696 lines HTML/CSS/JS

Tests: ~468 lines across 5 test modules · **29 tests, 100% passing**

3. Key Components

3.1 Data Agent

Loads one of four built-in scikit-learn datasets (iris, breast_cancer, wine, diabetes) or any user-supplied CSV. It automatically: - Infers the task type (classification if target has ≤ 10 unique values, otherwise regression) - Profiles the dataset (row/column counts, feature types, missing values, target distribution) - Creates a stratified 80/20 train/test split (stratified for classification, random for regression)

3.2 Feature Agent

Plans and builds a scikit-learn ColumnTransformer preprocessing pipeline: - **Numeric features:** median imputation → standard scaling - **Categorical features:** most-frequent imputation → one-hot encoding

Exposes both `plan()` / `build_preprocessor()` separately (so the plan can be reused by multiple agents) and a `plan_and_build()` convenience method for single-call use.

3.3 Model Search Agent

Trains 4 candidate models in **parallel** using `ProcessPoolExecutor`:

Task	Candidates
Classification	Logistic Regression, Random Forest, Extra Trees, SVC (RBF)
Regression	Ridge, Random Forest, Extra Trees, SVR (RBF)

Sets `n_jobs=1` per candidate to avoid daemon-process conflicts, instead parallelising at the model level (one process per candidate). Supports filtering to a subset of models via `--models` CLI flag.

3.4 Evaluation Agent

Computes task-appropriate metrics on the held-out test set: - **Classification:** accuracy, F1-macro, ROC-AUC (binary only) - **Regression:** RMSE, MAE, R^2

Ranks candidates by the primary metric (`f1_macro` for classification, `rmse` for regression) using a shared `is_higher_better()` method to eliminate any duplicate direction logic.

3.5 Hyperparameter Agent

Tunes the best candidate using **Optuna** (TPE sampler) with configurable trial counts. Falls back to RandomizedSearchCV if Optuna is unavailable. Uses StratifiedKFold for classification and KFold for regression. Tuning can be skipped entirely with `--trials 0`.

3.6 Explainability Agent

Computes **permutation importance** on the test set (`n_repeats=5`) and returns the top 12 features ranked by mean importance drop in the primary metric. Results are stored both as a JSON artifact and inside the model bundle for serving-time access.

3.7 Monitoring Agent

Builds a baseline from training data statistics: - **Numeric**: mean, std, min, max, missing rate - **Categorical**: top-20 value distribution, missing rate

At serving time, `check_drift()` compares incoming rows against the baseline: - **Numeric**: flags features where the observed mean shifts more than `drift_threshold_z` standard deviations (default 3.0, configurable via CLI) - **Categorical**: flags features containing unseen category values

3.8 Deployment Agent

Packages the final model into a self-contained **joblib bundle** containing the full sklearn pipeline, metadata, metrics, explainability report, and monitoring baseline. Writes a `manifest.json` with paths relative to the output directory (portable if moved) and registers the run in an append-only `registry.json`.

4. REST API & Frontend

4.1 API Endpoints

The FastAPI application (`uvicorn automl_agent.serving.app:app`) exposes:

Method	Route	Auth	Description
GET	/	Public	Browser console UI
GET	/health	Public	Server + bundle status
GET	/auth/status	Public	Auth configuration
GET	/auth/login	Public	Google OAuth redirect
GET	/auth/callback	Public	OAuth callback
POST	/auth/logout	Optional	Clear session
GET	/auth/me	Required	Current user info
GET	/schema	Optional	Feature schema + metrics
GET	/metadata	Optional	Full model metadata
POST	/predict	Optional	Run model predictions

Method	Route	Auth	Description
POST	/drift	Optional	Check input for drift

4.2 Browser Console

A single-page application served directly by FastAPI (no separate build step). Four panels: - **Overview** — model name, version, task, metrics, dataset profile - **Predict** — JSON textarea input, prediction + probability output - **Drift** — compare input rows against training baseline - **Explainability** — feature importance bar chart

4.3 Google Authentication

Optional OAuth2/OIDC integration via Authlib. When enabled: - Session signed with itsdangerous (requires SESSION_SECRET_KEY $\geq$ 32 characters) - Email verification enforced - Domain allowlist via GOOGLE_ALLOWED_DOMAINS env var - All protected endpoints return 401 without a valid session

5. Software Engineering Principles

5.1 SOLID Principles

Principle	Application
Single Responsibility	Each agent owns exactly one pipeline step; orchestrator.py owns sequencing; serving/ owns API concerns
Open/Closed	New model families added in _candidates() without touching the orchestrator; new auth providers added as separate modules
Liskov Substitution	ModelBundleStore can be swapped for a remote store implementing the same public interface
Interface Segregation	Serving config (GoogleAuthSettings, ServingSettings) and pipeline types (DataBundle, FeaturePlan) are kept separate
Dependency Inversion	create_app() accepts explicit settings and store instances for testing and alternate deployments

5.2 Other Principles

- **DRY** — EvaluationAgent.is_higher_better() is the single source of truth for metric direction, used by both rank() and orchestrator._choose_final()

- **KISS** — standard Python, scikit-learn pipelines, FastAPI, Authlib — no custom frameworks
- **YAGNI** — no distributed scheduler, no cloud store, no database until the use case needs them
- **Fail Fast** — missing bundles and incomplete auth config raise errors at startup, not at request time
- **Least Privilege** — /health and /auth/status are always public; all model endpoints require login when auth is enabled

6. Testing

6.1 Test Suite Summary

Module	Tests	What it covers
test_agents.py	15	FeatureAgent plan/build, MonitoringAgent numeric & categorical drift, ModelRegistry append/prune, PredictRequest validation
test_serving_auth.py	7	Auth on/off behaviour, credential validation, session key enforcement, env var parsing, dependency injection
test_harness.py	3	Multi-case execution, config file loading, failed case handling
test_pipeline.py	2	End-to-end classification (iris) and regression (diabetes) pipelines
test_frontend.py	2	Static asset serving, public /auth/status endpoint
Total	29	100% passing

6.2 Test Strategy

- **End-to-end tests** (test_pipeline.py) validate the full 8-step pipeline produces a loadable bundle and correct artifacts
- **Unit tests** (test_agents.py) isolate individual agent logic — especially edge cases like the categorical drift branch and registry pruning — without running the full pipeline
- **Integration tests** (test_serving_auth.py, test_frontend.py) verify the FastAPI app behaves correctly with and without auth, using TestClient with

injected settings

- **Harness tests** (test_harness.py) verify aggregate output files (JSON, CSV, Markdown) and failure handling

7. Security

Concern	Mitigation
Session forgery	itsdangerous-signed session cookie; SESSION_SECRET_KEY enforced ≥ 32 characters at startup
Unverified Google accounts	email_verified checked before accepting any login; email normalised to lowercase for domain comparison
Domain restriction	GOOGLE_ALLOWED_DOMAINS env var; both email domain and OIDC hd claim are checked
Resource exhaustion	/predict and /drift capped at 1,000 rows per request (Pydantic max_length)
CORS	CORSMiddleware added; origins configured via CORS_ORIGINS env var (empty = same-origin only)
Missing columns	Explicit 422 response listing which features are absent before reaching model inference

8. Experiment Harness

The ExperimentHarness enables repeatable, multi-dataset benchmarking. It: - Runs any number of named cases sequentially - Catches and records per-case failures without stopping the run (unless --fail-fast) - Writes three output formats after every case: results.json, results.csv, summary.md

Example harness.json:

```
{
  "cases": [
    {"name": "iris-fast", "dataset": "iris", "workers": 2, "trials": 0},
    {"name": "diabetes-reg", "dataset": "diabetes", "workers": 2, "trials": 0}
  ]
}
```

Run with:

```
automl-agent harness --config harness.json --output artifacts/harness
```

9. CLI Reference

```
automl-agent run
  --dataset      Built-in dataset (iris / wine / breast_cancer / diabetes)
  --csv          Path to a CSV file
  --target       Target column name (for CSV)
  --task         classification | regression (overrides inference)
  --output       Artifact output directory (default: artifacts/run)
  --workers      Parallel training workers (default: 4)
  --trials       Optuna tuning trials (default: 20; 0 = skip)
  --drift-threshold-z Z-score alert threshold (default: 3.0)
  --models       Comma-separated model names to train
```

```
automl-agent registry --path artifacts/registry.json
```

```
automl-agent harness --config harness.json --output artifacts/harness
```

10. Results

Running the full pipeline on the **Iris** dataset (classification):

Model	Accuracy	F1-Macro	Train Time
svc_rbf (winner)	96.7%	96.7%	—
extra_trees	96.7%	96.6%	—
random_forest	93.3%	93.2%	—
logistic_regression	90.0%	89.9%	—

Feature importance (permutation, test set):

Feature	Importance
petal width (cm)	0.3223 ± 0.073
petal length (cm)	0.2368 ± 0.042
sepal width (cm)	0.0472 ± 0.017
sepal length (cm)	0.0000 ± 0.000

Petal dimensions dominate — consistent with the botanical literature on iris classification.

11. Limitations & Future Work

Limitation	Potential Extension
Single-machine training only	Replace <code>ProcessPoolExecutor</code> with a distributed task queue (Celery, Ray)
Local JSON registry	Replace <code>ModelRegistry</code> with a database-backed registry (PostgreSQL, SQLite) while preserving the append-only contract
No experiment tracking	Integrate MLflow or Weights & Biases via a thin adapter over <code>HarnessResult</code>
No model comparison UI	Add a leaderboard view to the frontend console
No retraining trigger	Add a scheduled harness run or drift-triggered retraining hook
Fixed preprocessing	Support custom feature engineering steps via a plugin interface on <code>FeatureAgent</code>

12. Conclusion

The AutoML Agent System delivers a complete, working ML pipeline that goes from raw data to a served, monitored model in a single command. The architecture applies SOLID principles throughout: agents are independently testable, the orchestrator is easily extended, and the serving layer is environment-configurable with no hardcoded paths or credentials.

The 29-test suite provides confidence across the full stack — from individual agent unit tests through end-to-end pipeline runs to API authentication flows. The code remains intentionally lean (~1,634 lines of Python) without sacrificing correctness or extensibility.

Report generated from live codebase — all test results, metrics, and line counts reflect the current state of the repository.